12

Fast linear algebra

The "classical" algorithms for problems in linear algebra, such as matrix multiplication, computing the determinant, or solving systems of linear equations, all take $O(n^3)$ arithmetic operations for inputs of size $n \times n$. In this chapter, we discuss two totally different approaches to improving this. The first is a general method whose most powerful variant leads to $O(n^{2.3727})$ operations, but whose practical use may be limited. The second one uses a radically different model of linear algebra: instead of writing down the matrix ("explicit linear algebra"), we only use a (fast) mechanism for evaluating the matrix at a vector ("black box linear algebra"). This is only applicable with profit to restricted classes of matrices, but many problems arising in practice fall into this category: Sylvester, Vandermonde, and Toeplitz matrices, the Berlekamp matrix in Berlekamp's polynomial factorization algorithm 14.31, and large sparse matrices over $\mathbb{F}_2$ in integer factorization algorithms (Algorithm 19.12).

12.1. Strassen's matrix multiplication

Let R be a ring and $A, B \in R^{n \times n}$ two square matrices. The classical algorithm for computing the product matrix AB by computing n^2 row-by-column products uses n^3 multiplications and $n^3 - n^2$ additions in R. After having seen in Chapter 8 that we can multiply polynomials faster than by the obvious method, a natural question is whether something similar applies to matrix multiplication.

A few years after Karatsuba's polynomial and integer multiplication algorithm, Strassen (1969) found such an algorithm. The input matrices are divided into four $n/2 \times n/2$ blocks, and the computation of AB is reduced to seven multiplications and 18 additions of $n/2 \times n/2$ matrices, in comparison to eight multiplications and four additions for the classical algorithm. Like in the polynomial case, the multiplications are handled recursively, and the saving of one (costly) multiplication at the expense of 14 (cheap) additions leads to an asymptotically smaller running time of $O(n^{\log_2 7})$ operations in R, where the exponent is $\log_2 7 = 2.807354922\ldots$

https://doi.org/10.1017/CBO9781139856065.015 Published online by Cambridge University Press

We present a slightly different version, also using seven multiplications of $n/2 \times n/2$ matrices, but only 15 additions.

ALGORITHM 12.1 Matrix multiplication.
Input: $A, B \in R^{n\times n}$, where R is a ring and $n = 2^k$ for some $k \in \mathbb{N}$.
Output: The product matrix $AB \in R^{n\times n}$.

1. **if** $n = 1$ **then** let $A = (a)$, $B = (b)$ for some $a, b \in R$ and **return** (ab)
2. write $A = \begin{pmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{pmatrix}$, $B = \begin{pmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{pmatrix}$, with all $A_{ij}, B_{ij} \in R^{(n/2)\times(n/2)}$
3. $$\begin{aligned} S_1 &\longleftarrow A_{21} + A_{22}, & T_1 &\longleftarrow B_{12} - B_{11} \\ S_2 &\longleftarrow S_1 - A_{11}, & T_2 &\longleftarrow B_{22} - T_1 \\ S_3 &\longleftarrow A_{11} - A_{21}, & T_3 &\longleftarrow B_{22} - B_{12} \\ S_4 &\longleftarrow A_{12} - S_2, & T_4 &\longleftarrow T_2 - B_{21} \end{aligned}$$
4. **call** the algorithm recursively to compute
$$\begin{aligned} P_1 &= A_{11}B_{11}, & P_5 &= S_1T_1, \\ P_2 &= A_{12}B_{21}, & P_6 &= S_2T_2, \\ P_3 &= S_4B_{22}, & P_7 &= S_3T_3. \\ P_4 &= A_{22}T_4, \end{aligned}$$
5. $$\begin{aligned} U_1 &\longleftarrow P_1 + P_2, & U_5 &\longleftarrow U_4 + P_3 \\ U_2 &\longleftarrow P_1 + P_6, & U_6 &\longleftarrow U_3 - P_4 \\ U_3 &\longleftarrow U_2 + P_7, & U_7 &\longleftarrow U_3 + P_5 \\ U_4 &\longleftarrow U_2 + P_5 \end{aligned}$$
6. **return** $\begin{pmatrix} U_1 & U_5 \\ U_6 & U_7 \end{pmatrix}$

THEOREM 12.2.
Algorithm 12.1 correctly computes the product matrix and uses at most $6n^{\log_2 7}$ *additions and multiplications in* R. *For arbitrary* $n \in \mathbb{N}$, *an* $n \times n$ *matrix product can be computed with* $42n^{\log_2 7} \in O(n^{\log_2 7})$ *ring operations.*

PROOF. The correctness is left as Exercise 12.1. For $n = 2^k \in \mathbb{N}$, let $T(n)$ denote the number of arithmetic operations in R that the algorithm performs on inputs of size $n \times n$. Then $T(1) = 1$ and $T(2^k) = 15 \cdot 2^{2k-2} + 7T(2^{k-1})$ for $k \geq 1$, and Lemma 8.2 implies the first claim. For arbitrary n, we pad the matrices with zeroes, thereby at most doubling the dimension. □

https://doi.org/10.1017/CBO9781139856065.015 Published online by Cambridge University Press

Strassen's (1969) discovery was the starting signal for the development of fast algorithms. Although subquadratic integer multiplication algorithms had been around for a while (Section 8.1), it was the surprise of realizing that the "obvious" cubic algorithm for matrix multiplication could be improved that kicked this development into high gear and inspired, within the following five years, the many new ideas for almost all the fast algorithms discussed in Part II of this book.

On a more technical level, Strassen's result spawned three lines of research:

- faster matrix multiplication,
- other problems from linear algebra,
- bilinear complexity.

For a field F, a number $\omega \in \mathbb{R}$ is a **feasible matrix multiplication exponent** if two $n \times n$ matrices over F can be multiplied with $O(n^\omega)$ operations in F. The classical algorithm shows that $\omega = 3$ is feasible, and Strassen's that $\omega = \log_2 7$ is. The **matrix multiplication exponent** μ (for F) is the infimum of all feasible ones. Thus

$$2 \leq \mu \leq \omega$$

for all feasible ω's. This μ is the same for all fields of a fixed characteristic, and all feasible exponents discovered so far work for all fields.

The fascinating history of the smallest exponents known is in the Notes 12.1; the current world record is $\omega < 2.3727$. It seems natural to conjecture that $\mu = 2$; there is currently no method in sight that might prove or disprove this.

How practical are these algorithms? Bailey, Lee & Simon (1990) deplore "an unfortunate myth [...] regarding the *crossover* point for Strassen's algorithm", and show that for the Sun–4 "Strassen is faster for matrices as small as 16×16. For Cray systems the crossover point is roughly 128". They conclude that "it appears that Strassen's algorithm can indeed be used to accelerate practical-sized linear algebra calculations." Besides a Cray library implementation (SGEMMS) of fast matrix multiplication, there is also one, the ESSL library, for IBM 3090 machines. Higham (1990) reports on a set of FORTRAN 77 routines (level 3 BLAS) using "Strassen's method for fast matrix multiplication, which is now recognized to be a practically useful technique once matrix dimensions exceed about 100". In all these experiments, the coefficients are floating point numbers of a fixed precision.

A further avenue to explore with Strassen's algorithm is that its recursive partition employs data access that is essentially different from classical multiplication. This may make it attractive for machines with a hierarchical memory structure and for large matrices stored in secondary memory, possibly reducing data transfer (paging) time.

Further computational problems in linear algebra include matrix inversion, computing the determinant, the characteristic polynomial, or the LR-decomposition of a matrix, and, for $F = \mathbb{C}$, the QR-decomposition and unitary transformation to

https://doi.org/10.1017/CBO9781139856065.015 Published online by Cambridge University Press

upper Hessenberg form. It turns out that all these problems have the same asymptotic complexity as matrix multiplication (up to constant factors), so that a fast algorithm for one of them immediately gives fast algorithms for all of them.

The exponent η for solving systems of linear equations satisfies $\eta \leq \omega$ for all feasible ω. It is not known whether $\eta = \mu$.

The most fundamental consequence of Strassen's breakthrough was the development of *bilinear complexity theory*, a deep and rich area that is concerned with good and optimal algorithms for functions that depend linearly on each of two sets of variables, just like the entries of the product of two matrices (or polynomials) do. Bürgisser, Clausen & Shokrollahi (1997) give a detailed account of the achievements in this theory, which is part of *algebraic complexity theory*.

12.2. Application: fast modular composition of polynomials

As an application of fast matrix multiplication, we now discuss a fast algorithm for **modular composition** of polynomials. The problem is, given three polynomials $f, g, h \in R[x]$ with $\deg g, \deg h < \deg f = n$, where R is a (commutative) ring and $f \neq 0$ is monic, to compute $g(h) \bmod f$, that is, the remainder of the composition of g and h modulo f. Using Horner's rule, this can be done with at most n multiplications and additions modulo f, at a cost of $O(n\mathsf{M}(n))$ operations in R. The surprising fact is that we can do better if we use not only fast polynomial arithmetic but also fast matrix arithmetic.

ALGORITHM 12.3 Fast modular composition.
Input: $f, g, h \in R[x]$ with $\deg g, \deg h < \deg f = n$ and $f \neq 0$ monic.
Output: $g(h) \bmod f \in R[x]$.

1. $m \longleftarrow \lceil n^{1/2} \rceil$
   let $g = \sum_{0 \leq i < m} g_i x^{mi}$, with $g_0, \ldots, g_{m-1} \in R[x]$ of degree less than m
2. **for** $i = 2, \ldots, m$ compute $h^i \bmod f$
3. let $A \in R^{m \times n}$ be the matrix whose rows are the coefficients of 1, $h \bmod f$, $h^2 \bmod f$, …, $h^{m-1} \bmod f$, and $B \in R^{m \times m}$ the matrix whose rows are the coefficients of $g_0, g_1, \ldots, g_{m-1}$, and compute $BA \in R^{m \times n}$ via $\lceil n/m \rceil \leq m$ matrix multiplications of size $m \times m$
4. **for** $i = 0, \ldots, m-1$ **do**
   let $r_i \in R[x]$ be the polynomial whose coefficients form the ith row of BA, and compute $b = \sum_{0 \leq i < m} r_i \cdot (h^m)^i \bmod f$ using Horner's rule
5. **return** b

https://doi.org/10.1017/CBO9781139856065.015 Published online by Cambridge University Press

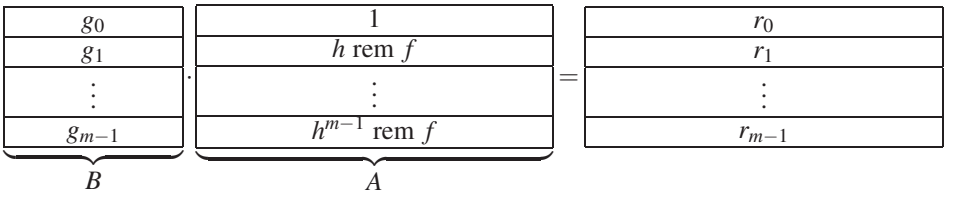


FIGURE 12.1: The matrix product in the modular composition algorithm 12.3.

THEOREM 12.4.
Algorithm 12.3 works correctly and uses at most $\lceil n^{1/2} \rceil$ matrix multiplications of size $\lceil n^{1/2} \rceil \times \lceil n^{1/2} \rceil$, plus no more than $6n^{1/2}(\mathsf{M}(n) + O(n))$ additions and multiplications in R.

PROOF. Let $i < m$, and $g_i = \sum_{0 \le j < m} g_{ij} x^j$ with all $g_{ij} \in R$. Then $r_i = \sum_{0 \le j < m} g_{ij} \cdot (h^j \text{ rem } f) = g_i(h) \text{ rem } f$ (see Figure 12.1), and

$$b \equiv \sum_{0 \le i < m} r_i \cdot (h^m)^i \equiv \sum_{0 \le i < m} g_i(h) \cdot h^{mi} = g(h) \bmod f.$$

The cost for step 2 is $m - 1$ multiplications modulo f. We may precompute $\text{rev}_n(f)^{-1} \bmod x^n$ for the fast division algorithm, at a cost of $3\mathsf{M}(n) + O(n)$, and then step 2 takes $3n^{1/2}\mathsf{M}(n) + O(n^{3/2})$ ring operations, by Corollary 9.7 and the discussion following it. Step 3 may be done by computing about m products of pairs of $m \times m$ matrices. Finally, step 4 costs at most m multiplications and additions modulo h, with about the same cost as for step 2. □

Taking $\mathsf{M}(n) \in O(n \log n \operatorname{loglog} n)$ and $\omega < 2.3727$, we obtain the following.

COROLLARY 12.5.
The modular composition $g(h)$ rem f for three polynomials $f, g, h \in R[x]$ with $\deg g, \deg h < \deg f = n$ and $f \neq 0$ monic can be computed using $O(n^{1.687})$ operations in R.

Umans (2008) and Kedlaya & Umans (2009) present major progress, namely a new algorithm for modular composition that uses only $n^{1+o(1)} \cdot O^{\sim}(\log q)$ bit operations. This corresponds, up to small factors, to only $O(n)$ operations in $\mathbb{F}_q$. One of the main innovations in their work is that they do not use algebra in $\mathbb{F}_q$, but bit operations. Namely, they reduce the problem to multipoint evaluation for multivariate polynomials. They solve this task in a modular fashion, by considering it over the integers, solving it modulo various primes, and reconstructing the result via Chinese remaindering.

https://doi.org/10.1017/CBO9781139856065.015 Published online by Cambridge University Press

12.3. Linearly recurrent sequences

In this and the following section, we introduce a new way, called **black box linear algebra**, of solving linear equations. So the task is, for a given matrix $A \in F^{n\times m}$ and a vector $b \in F^n$ over a field F, to find one (or a description of all) solution $y \in F^m$ to the equation $Ay = b$. From basic linear algebra, we know that Gaussian elimination solves the problem with $O(\max\{n,m\}^3)$ operations in F (Section 25.5).

In some applications, such as polynomial factorization (Section 14.8) and integer factorization (Section 19.5), linear systems have to be solved where the coefficient matrix has a special form, for example, it is sparse. Wiedemann (1986) presented an algorithm that is asymptotically faster than Gaussian elimination for sparse matrices. Kaltofen & Saunders (1991) gave a more general variant of Wiedemann's algorithm that is particularly well suited for a much larger class of matrices, namely those where multiplication of the matrix by an arbitrary vector is cheap. A square matrix $A \in F^{n\times n}$ is not given by its $n \times n$ array of entries, but by a black box for evaluating A, that is, a procedure which on input of a vector $v \in F^n$ produces $Av \in F^n$. We denote by $c(A)$ the arithmetic cost for evaluating A at a vector. Then the cost of Kaltofen & Saunders' version of Wiedemann's algorithm is essentially $O(n \cdot c(A))$. If A is arbitrary and we use the classical algorithm for computing Av, then $c(A)$ is about $2n^2$, and Wiedemann's algorithm provides no asymptotic gain over Gaussian elimination, but it does if A is special such that $c(A) \in o(n^2)$. Table 12.2 collects the evaluation costs for some important classes of matrices.

class of matrices	$c(A)$
general	$2n^2 - n$
Sylvester matrix	$O(\mathsf{M}(n))$
DFT_ω, ω a primitive 2^kth root of unity	$O(n \log n)$
Vandermonde matrix	$O(\mathsf{M}(n) \log n)$
Berlekamp matrix over $\mathbb{F}_q$	$O(\mathsf{M}(n) \log q)$
sparse, with $\leq s$ nonzero entries	$\leq 2s$
random squares integer factorization	$O(n \log^2 n)$

TABLE 12.2: Evaluation cost for some matrix classes.

For matrices with "small" evaluation cost we also have "fast" matrix multiplication, in that the product with an arbitrary (explicitly given) $n \times n$ matrix can be calculated with $n \cdot c(A)$ arithmetic operations. The *transposition principle* (see Notes 12.3) says that for the evaluation cost, it does not matter whether we multiply by a vector from the right or the left, or whether we consider a matrix or its transpose.

https://doi.org/10.1017/CBO9781139856065.015 Published online by Cambridge University Press

Before we can present the algorithm, we need some facts about linearly recurrent sequences. Let F be a field and $V \neq \{0\}$ a vector space over F. Then $V^{\mathbb{N}}$ is the (infinite-dimensional) vector space of infinite sequences $(a_i)_{i\in\mathbb{N}}$, with all $a_i \in V$.

DEFINITION 12.6. *A sequence $a = (a_i)_{i\in\mathbb{N}} \in V^{\mathbb{N}}$ is **linearly recurrent (over F)** if there exist $n \in \mathbb{N}$ and $f_0, \ldots, f_n \in F$ with $f_n \neq 0$ such that*

$$\sum_{0\leq j\leq n} f_j a_{i+j} = f_n a_{i+n} + \cdots + f_1 a_{i+1} + f_0 a_i = 0$$

*for all $i \in \mathbb{N}$. The polynomial $f = \sum_{0\leq j\leq n} f_j x^j \in F[x]$ of degree n is called a **characteristic** (or annihilating, or generating) **polynomial** of a.*

EXAMPLE 12.7. (i) $V = F$, $a_i = 0$ for all $i \in \mathbb{N}$. This sequence is linearly recurrent, and *any* nonzero polynomial f is a characteristic polynomial.

(ii) $V = F = \mathbb{Q}$, $a_0 = 0$, $a_1 = 1$, $a_{i+2} = a_{i+1} + a_i$ for all $j \geq 2$. Then $a = (a_i)_{i\geq 0}$ is the **Fibonacci sequence**, and $f = x^2 - x - 1$ is a characteristic polynomial of a.

(iii) $V = F^{n\times n}$, $A \in F^{n\times n}$ arbitrary. Then $a = (A^i)_{i\geq 0}$ is linearly recurrent, and the characteristic polynomial $\chi_A \in F[x]$ is a characteristic polynomial of a, by the Cayley–Hamilton theorem (Section 25.5).

(iv) $V = F^n$, $A \in F^{n\times n}$ and $b \in F^n$ arbitrary. Then $a = (A^i b)_{i\in\mathbb{N}}$ is linearly recurrent, and any characteristic polynomial of $(A^i)_{i\in\mathbb{N}}$ also annihilates a. The subspace of F^n spanned by the entries of a is called the **Krylov subspace of A and b**.

(v) $V = F$, $A \in F^{n\times n}$, $b, u \in F^n$ arbitrary. Then $a = (u^T A^i b)_{i\in\mathbb{N}}$ is linearly recurrent, and any characteristic polynomial of $(A^i b)_{i\in\mathbb{N}}$ also annihilates a.

(vi) This generalizes the two previous examples. If V and W are vector spaces over F, $\varphi: V \longrightarrow W$ is F-linear, and $a = (a_i)_{i\in\mathbb{N}} \in V^{\mathbb{N}}$ is linearly recurrent with characteristic polynomial $f \in F[x]$, then so is $\varphi(a) = (\varphi(a_i))_{i\in\mathbb{N}} \in W^{\mathbb{N}}$. $\diamond$

A linearly recurrent sequence $a = (a_i)_{i\in\mathbb{N}} \in V^{\mathbb{N}}$ with characteristic polynomial $f = \sum_{0\leq j\leq n} f_j x^j \in F[x]$ of degree n is completely determined by the n initial values $a_0, \ldots, a_{n-1}$. The ith entry of a can be computed in linear time with space for n elements of F, independent of i, using a **linear feedback shift register** (see Figure 12.3). It consists of a shift register with n places, holding n consecutive entries $a_i, \ldots, a_{i+n-1}$ of a at a time, plus n gates for multiplication by the f_j and $n+1$ addition gates. The shift register is synchronized by an external clock. At the ith clock tick, the entries of the shift register are shifted one place to the left, the leftmost entry a_i is output, and the next entry $a_{i+n} = \sum_{0\leq j<n} f_j a_{i+j}$ of the sequence is computed by the part of the circuit consisting of arithmetic gates, and fed into the rightmost place of the shift register. Initially, the shift register is loaded with $a_0, \ldots, a_{n-1}$.

https://doi.org/10.1017/CBO9781139856065.015 Published online by Cambridge University Press

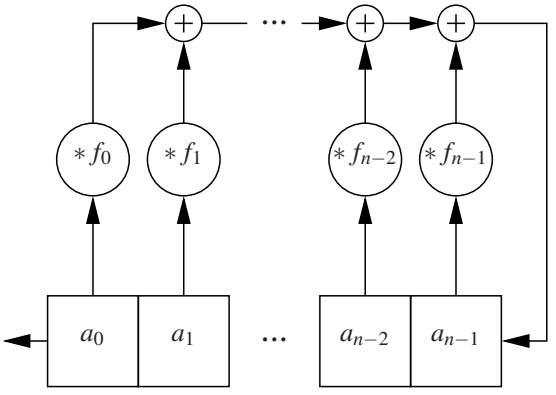


FIGURE 12.3: Initial state of a linear feedback shift register for the sequence $a = (a_i)_{i\in\mathbb{N}}$ with characteristic polynomial $f = x^n - f_{n-1}x^{n-1} - f_{n-2}x^{n-2} - \cdots - f_1x - f_0$.

It is convenient to define a multiplication of sequences by polynomials. For $f = \sum_{0\le j\le n} f_j x^j \in F[x]$ and $a = (a_i)_{i\in\mathbb{N}} \in V^{\mathbb{N}}$, we set

$$f \bullet a = \Big(\sum_{0\le j\le n} f_j a_{i+j} \Big)_{i\in\mathbb{N}} \in V^{\mathbb{N}}.$$

The constants $f \in F$ act on sequences in the usual way, and the indeterminate x acts as a shift operator:

$$x \bullet a = (a_{i+1})_{i\in\mathbb{N}}.$$

This makes $V^{\mathbb{N}}$, together with $\bullet$, into an $F[x]$-module. A module is something similar to a vector space, with the only difference that the "scalars" may be elements of an arbitrary (commutative) ring instead of a field. In particular, $\bullet$ has the following properties:

$$f \bullet (a+b) = f \bullet a + f \bullet b, \tag{1}$$
$$f \bullet \mathbf{0} = \mathbf{0}, \tag{2}$$
$$(f+g) \bullet a = f \bullet a + g \bullet a, \tag{3}$$
$$(fg) \bullet a = f \bullet (g \bullet a) = g \bullet (f \bullet a), \tag{4}$$
$$0 \bullet a = \mathbf{0}, \tag{5}$$
$$1 \bullet a = a, \tag{6}$$

for all $f, g \in F[x]$ and $a, b \in V^{\mathbb{N}}$, where $\mathbf{0} = (0)_{i\in\mathbb{N}}$ is the zero sequence. Their proof is in Exercise 12.5. For example, every commutative group G is a $\mathbb{Z}$-module by letting $f \bullet a = a^f$ for $a \in G$ and $f \in \mathbb{Z}$.

We can express the property of being a characteristic polynomial in terms of the operation $\bullet$: $f \in F[x] \setminus \{0\}$ is a characteristic polynomial of $a \in V^{\mathbb{N}}$ if and only if $f \bullet a = \mathbf{0}$. The set of all characteristic polynomials of a sequence $a \in V^{\mathbb{N}}$,

https://doi.org/10.1017/CBO9781139856065.015 Published online by Cambridge University Press

together with the zero polynomial, is an ideal in $F[x]$: if f, g are both characteristic polynomials or zero, then so is $f+g$, and if $r \in F[x]$ is arbitrary, then rf is either zero or a characteristic polynomial, by (2), (3), and (4). This ideal is called the **annihilator** of a and denoted by $\mathrm{Ann}(a)$. Since any ideal in $F[x]$ is generated by a single polynomial (Section 25.3), either $\mathrm{Ann}(a) = \{0\}$ or there is a unique monic polynomial $m \in \mathrm{Ann}(a)$ of least degree such that $\langle m \rangle = \{rm : r \in F[x]\} = \mathrm{Ann}(a)$. This polynomial is called the **minimal polynomial** of a and divides any other characteristic polynomial of a. We denote it by m_a. If a is not linearly recurrent, then $\mathrm{Ann}(a) = \{0\}$, and we set $m_a = 0$. The degree of m_a is called the **recursion order** of a. Summarizing, we have the following equivalences for $f \in F[x]$ and $a \in V^{\mathbb{N}}$:

$$\begin{gathered} f = 0 \text{ or } f \text{ is a characteristic polynomial of } a \iff f \bullet a = \mathbf{0} \\ \iff f \in \mathrm{Ann}(a) \iff m_a \mid f, \\ a \in V^{\mathbb{N}} \text{ is linearly recurrent} \iff \exists f \in F[x] \setminus \{0\} \quad f \bullet a = \mathbf{0} \\ \iff \mathrm{Ann}(a) \neq \{0\} \iff m_a \neq 0. \end{gathered}$$

EXAMPLE 12.7 (continued). (i) Any polynomial annihilates the zero sequence, by (2). Thus $\mathrm{Ann}(\mathbf{0}) = F[x]$ and $m_{\mathbf{0}} = 1$.

(ii) The minimal polynomial of the Fibonacci sequence is $m_a = x^2 - x - 1$. This is because the polynomial is irreducible over $\mathbb{Q}$ (its roots $(1 \pm \sqrt{5})/2$ are irrational), and hence no proper divisor of m_a annihilates a (1 obviously does not).

(iii) The minimal polynomial of the matrix A is also the minimal polynomial of the sequence $(A^i)_{i \in \mathbb{N}}$.

(iv) m_a divides the minimal polynomial of A.

(v) m_a divides the minimal polynomial of $(A^i b)_{i \in \mathbb{N}}$.

(vi) $\mathrm{Ann}(a) \subseteq \mathrm{Ann}(\varphi(a))$ and $m_{\varphi(a)} \mid m_a$.

(vii) Let V be an algebraic field extension of F, $\alpha \in V$, and $a = (\alpha^i)_{i \geq 0}$. Then a is linearly recurrent, and the minimal polynomial of a is the minimal polynomial of α over F. $\diamond$

We now indicate how to compute the minimal polynomial of a given sequence $a = (a_i)_{i \in \mathbb{N}} \in F^{\mathbb{N}}$, provided that we know an upper bound $n \in \mathbb{N}$ on the recursion order. We recall from Section 9.1 the reversal of a polynomial: for $f = f_d x^d + \cdots + f_0 \in F[x]$ of degree d, we have

$$\mathrm{rev}(f) = \mathrm{rev}_d(f) = x^d f(x^{-1}) = f_0 x^d + f_1 x^{d-1} + \cdots + f_d \in F[x].$$

LEMMA 12.8. *Let $a = (a_i)_{i \in \mathbb{N}} \in F^{\mathbb{N}}$ be linearly recurrent, $h = \sum_{i \in \mathbb{N}} a_i x^i \in F[[x]]$, the formal power series whose coefficients are the coefficients of the sequence a, $f \in F[x] \setminus \{0\}$ of degree d, and $r = \mathrm{rev}(f)$ its reversal.*

https://doi.org/10.1017/CBO9781139856065.015 Published online by Cambridge University Press

(i) The following are equivalent.

(a) f is a characteristic polynomial of a,

(b) $r \cdot h$ is a polynomial of degree less than d,

(c) $h = g/r$ for some $g \in F[x]$ with $\deg g < d$.

(ii) If f is the minimal polynomial of a, then $d = \max\{1 + \deg g, \deg r\}$ and $\gcd(g,r) = 1$ in (i).

PROOF. For the proof of (i), see Exercise 12.7. For (ii), we note that $\deg r \leq d$, with equality if and only if $x \nmid f$, and hence $d \geq \max\{1 + \deg g, \deg r\}$ in (i). Now let $f = m_a$, and suppose that $d > \max\{1 + \deg g, \deg r\}$. Then x divides f, $r = \operatorname{rev}(f/x)$, and f/x is a characteristic polynomial of a of degree $d-1$, by (i), contradicting the minimality of m_a. Thus $d = \max\{1 + \deg g, \deg r\}$.

Let $u = \gcd(g,r)$. Then $f^* = f/\operatorname{rev}(u)$ is a polynomial of degree $d - \deg u$, $r/u = \operatorname{rev}(f^*)$, and $(r/u)h = (g/u)$ is a polynomial of degree less than $d - \deg u$. Hence f^* is a characteristic polynomial of a, again by (i), and the minimality of d implies that $\deg u = 0$. □

If $n \in \mathbb{N}$ is an upper bound on the recursion order of a, then we may compute m_a by solving the Padé approximation problem

$$h \equiv \frac{s}{t} \bmod x^{2n}, \quad x \nmid t, \quad \deg s < n, \quad \deg t \leq n, \quad \gcd(s,t) = 1, \tag{7}$$

since Lemma 12.8 (ii) implies that $(s,t) = (g,r)$ is a solution to (7) (note that $x \nmid r$, by the definition of rev). We have seen in Section 5.9 that the solution to (7) is unique (up to multiplication by constants) and can be computed with the Extended Euclidean Algorithm, using $O(n^2)$ arithmetic operations in F. This leads to the following algorithm.

ALGORITHM 12.9 Minimal polynomial for $F^{\mathbb{N}}$.

Input: An upper bound $n \in \mathbb{N}$ on the recursion order and the first $2n$ entries $a_0, \ldots, a_{2n-1} \in F$ of a linearly recurrent sequence $a \in F^{\mathbb{N}}$.

Output: The minimal polynomial $m_a \in F[x]$ of a.

1. $h \longleftarrow a_{2n-1}x^{2n-1} + \cdots + a_1 x + a_0$
   call the Extended Euclidean Algorithm to compute $s,t \in F[x]$ such that $t(0) = 1$ and (7) holds, as described in Section 5.9

2. $d \longleftarrow \max\{1 + \deg s, \deg t\}$, **return** $\operatorname{rev}_d(t)$

https://doi.org/10.1017/CBO9781139856065.015 Published online by Cambridge University Press

THEOREM 12.10.
Algorithm 12.9 correctly computes the minimal polynomial of a linearly recurrent sequence $(a_i)_{i\in\mathbb{N}}$ of recursion order at most n and uses $O(n^2)$ operations in F.

PROOF. Let $f \in F[x]$ be the minimal polynomial of a. The discussion preceding the algorithm implies that $(g,r) = (s,t)$, where g,r are as in Lemma 12.8 (i). Finally, $f = \mathrm{rev}_k(r)$ for some $k \in \mathbb{N}$, and Lemma 12.8 (ii) implies that $k = d$. □

Using the fast Euclidean Algorithm from Chapter 11, the minimal polynomial can actually be computed with $O(\mathsf{M}(n)\log n)$ field operations, but this does not help in our intended application.

EXAMPLE 12.11. (i) Let $F = \mathbb{F}_5$ and $a = (3,0,4,2,3,0,\ldots) \in \mathbb{F}_5^{\mathbb{N}}$ be linearly recurrent of recursion order at most 3. Then $h = 3x^4 + 2x^3 + 4x^2 + 3$ in step 1 of Algorithm 12.9, and the relevant results of the Extended Euclidean Algorithm for x^6 and h are

j	q_{j-1}	a_j	t_j
0		x^6	0
1		$3x^4+2x^3+4x^2+3$	1
2	$2x^2+2x+1$	$4x+2$	$3x^2+3x+4$
3	$2x^3+2x^2$	3	$4x^5+3x^4+x^3+2x^2+1$
4	$3x+4$	0	$3x^6$

We read off row 2 that

$$h \equiv \frac{4x+2}{3x^2+3x+4} = \frac{x+3}{2x^2+2x+1} \bmod x^6,$$

whence $s = x+3$ and $t = 2x^2+2x+1$. Finally, in step 2 we have $d = 2$ and $m_a = \mathrm{rev}_2(t) = x^2+2x+2$. We check that indeed $a_{i+2}+2a_{i+1}+2a_i = 0$ in $\mathbb{F}_5$ for $i = 0,1,2,3$. Thus the series continues as $(3,0,4,2,3,0,4,2,3,0,\ldots)$.

(ii) Let $a = (0,0,1,0,1,0,\ldots) \in F^{\mathbb{N}}$ of recursion order at most 3. Then $h = x^4+x^2$, the Extended Euclidean Algorithm for x^6 and h computes

j	q_{j-1}	a_j	t_j
0		x^6	0
1		x^4+x^2	1
2	x^2-1	x^2	$-x^2+1$
3	x^2+1	0	x^4

so that $s = x^2$ and $t = -x^2+1$. Here $d = 3$ and $m_a = \mathrm{rev}_3(t) = x^3 - x$. Thus $a_{i+3} = a_{i+1}$ for all $i \geq 0$, $a_i = 0$ if i is odd, and $a_i = 1$ if $i > 0$ is even. Hence $(a_1,a_2,a_3,\ldots)$ is periodic with period 2, and a has a **preperiod** of length 1. ◇

https://doi.org/10.1017/CBO9781139856065.015 Published online by Cambridge University Press

12.4. Wiedemann's algorithm and black box linear algebra

The main idea of Wiedemann's (1986) algorithm for solving linear equations is as follows. For simplicity, we assume that $A \in F^{n\times n}$ is a nonsingular square matrix. Then for any $b \in F^n$, $y = A^{-1}b \in F^n$ is the unique solution of $Ay = b$. Suppose that $m = m_a = \sum_{0\le j\le d} m_j x^j \in F[x]$ is the minimal polynomial of the linearly recurrent sequence $a = (A^i b)_{i\in\mathbb{N}}$, that is, the unique monic polynomial of least degree such that $m \bullet a = \mathbf{0}$. Then in particular the first entry of $m \bullet a$ is zero, and

$$m(A)b = \sum_{0\le j\le d} m_j A^j b = 0 \text{ in } F^n. \tag{8}$$

Now m is a divisor of the minimal polynomial of A, which in turn divides χ_A, the characteristic polynomial of A, according to Example 12.7 (iii). The constant coefficient of χ_A is $\det A \neq 0$, since A is nonsingular, and hence also the constant coefficient $m_0 = m(0)$ of m is nonzero. Thus

$$A \cdot (-m_0^{-1}) \sum_{1\le j\le d} m_j A^{j-1} b = -m_0^{-1} \sum_{1\le j\le d} m_j A^j b = b,$$

and $y = -m_0^{-1} \sum_{1\le j\le d} m_j A^{j-1} b \in F^n$ is the required solution and can be computed in a Horner-like fashion (Section 5.2), using $d - 1 < n$ evaluations of A at a vector plus $O(n^2)$ field operations for additions of vectors and multiplications by scalars. We note that y belongs to the Krylov subspace of A and b.

This leads to the following algorithm.

ALGORITHM 12.12 Solving a nonsingular square linear system.
Input: A nonsingular matrix $A \in F^{n\times n}$ and a vector $b \in F^n$.
Output: $y = A^{-1}b \in F^n$.

1. compute the minimal polynomial $m \in F[x]$ of the linearly recurrent sequence $(A^i b)_{i\in\mathbb{N}} \in (F^n)^{\mathbb{N}}$
2. $h \longleftarrow -\dfrac{m - m(0)}{m(0)\cdot x} \in F[x]$, compute $y = h(A)b$ in a Horner-like fashion
3. **return** y

We have reduced our original problem of solving $Ay = b$ to computing the minimal polynomial m_a of $a = (A^i b)_{i\in\mathbb{N}}$; any small multiple of m_a would also do. We have seen at the end of the previous section how to compute minimal polynomials for sequences whose entries are field elements, but there is no obvious analog of that procedure when the sequence entries are n-dimensional vectors. The idea now is to choose vectors $u \in F^n$ randomly, compute the minimal polynomial $m \in F[x]$ of the linearly recurrent sequence $(u^T A^i b)_{i\in\mathbb{N}} \in F^{\mathbb{N}}$, using Algorithm 12.9, and then

https://doi.org/10.1017/CBO9781139856065.015 Published online by Cambridge University Press

check whether m is actually the minimal polynomial of a (in general, it divides m_a) by computing $m(A)b \in F^n$. If p is the probability of success, then the algorithm will be successful after $O(1/p)$ expected trials (Section 25.6).

The following algorithm even works for singular square matrices.

ALGORITHM 12.13 Minimal polynomial for Krylov subspaces.
Input: A matrix $A \in F^{n\times n}$ and a vector $b \in F^n$.
Output: The minimal polynomial $m \in F[x]$ of the sequence $(A^i b)_{i\in\mathbb{N}}$.

1. **if** $b = 0$ **then return** 1
2. choose a finite set $U \subseteq F$
3. choose $u \in U^n$ uniformly at random and compute $u^T A^i b \in F$ for $0 \leq i < 2n$
4. **call** Algorithm 12.9 to compute the minimal polynomial $m \in F[x]$ of the linearly recurrent sequence $(u^T A^i b)_{i\in\mathbb{N}} \in F^{\mathbb{N}}$, with recursion bound n
5. **if** $m(A)b = 0$ in F^n **then return** m **else goto** 3

EXAMPLE 12.14. Let $F = \mathbb{F}_5$,

$$A = \begin{pmatrix} 1 & 4 & 4 \\ 4 & 0 & 3 \\ 1 & 2 & 4 \end{pmatrix} \text{ and } b = \begin{pmatrix} 3 \\ 1 \\ 2 \end{pmatrix},$$

and suppose that we want to find $y \in F^3$ such that $Ay = b$. We have

$$Ab = \begin{pmatrix} 0 \\ 3 \\ 3 \end{pmatrix}, \quad A^2 b = A(Ab) = \begin{pmatrix} 4 \\ 4 \\ 3 \end{pmatrix}, \quad A^3 b = A(A^2 b) = \begin{pmatrix} 2 \\ 0 \\ 4 \end{pmatrix},$$

$$A^4 b = A(A^3 b) = \begin{pmatrix} 3 \\ 0 \\ 3 \end{pmatrix}, \quad A^5 b = A(A^4 b) = \begin{pmatrix} 0 \\ 1 \\ 0 \end{pmatrix}.$$

In step 3 of Algorithm 12.13, we choose $u = (1,0,0)^T \in F^3$, and obtain

$$(u^T A^i b)_{i\in\mathbb{N}} = (3,0,4,2,3,0,\ldots).$$

We have already seen in Example 12.11 that the minimal polynomial of this sequence is $m = x^2 + 2x + 2$. In step 5 of Algorithm 12.13, we calculate

$$m(A)b = A^2 b + 2Ab + 2b = \begin{pmatrix} 0 \\ 2 \\ 3 \end{pmatrix},$$

https://doi.org/10.1017/CBO9781139856065.015 Published online by Cambridge University Press

so that m is not the minimal polynomial of $(A^i b)_{i\in\mathbb{N}}$. We go back to step 3 and choose $u = (1,2,0)^T$ this time. Then $(u^T A^i b)_{i\in\mathbb{N}} = (0,1,2,2,3,2,\ldots)$, and Algorithm 12.9 yields the minimal polynomial $m = x^3 + 3x + 1$. Since the minimal polynomial of $(A^i b)_{i\in\mathbb{N}}$ is a monic multiple of m of degree at most 3, it equals m. We check this by calculating

$$m(A)b = A^3 b + 3Ab + b = \begin{pmatrix} 0 \\ 0 \\ 0 \end{pmatrix}.$$

Finally, in step 2 of Algorithm 12.12, we compute

$$h = -\frac{m - m(0)}{m(0)x} = 4x^2 + 2, \quad y = h(A)b = 4A^2 b + 2b = \begin{pmatrix} 2 \\ 3 \\ 1 \end{pmatrix},$$

and in fact $Ay = b$. $\diamond$

THEOREM 12.15.
An output returned by Algorithm 12.13 is correct. If it returns after k iterations, the cost is at most $2n\,c(A) + O(kn^2)$ operations in F.

PROOF. Let $a = (A^i b)_{i\in\mathbb{N}}$. Correctness is clear if $b = 0$, and we assume that b is nonzero. The polynomial m computed in step 4 of the algorithm is a divisor of m_a, according to Example 12.7 (v) on page 341. If the algorithm returns m in step 5, then $m(A)b = 0$, so that m is a characteristic polynomial of a and hence a multiple of m_a. Since both m and m_a are monic, we have $m = m_a$.

Steps 3 and 5 can be done in a Horner-like way, using $2n$ applications of A to a vector in total if the $2n$ vectors $b, Ab, \ldots, A^{2n-1}b \in F^n$ are precomputed and stored after step 2, and $3n$ evaluations of A per iteration otherwise, plus $O(n^2)$ field operations per iteration for additions of vectors, inner products, and multiplications by scalars. The cost for step 4 is $O(n^2)$ for each iteration. $\square$

It remains to prove that the condition in step 5 is true with reasonable probability, so that we expect to get an output after a small number of iterations. Thus we have to find a lower bound on the probability that for random $u \in U^n$, the polynomial m computed in step 4 of Algorithm 12.13 is the minimal polynomial of $(A^i b)_{i\in\mathbb{N}}$.

For a nonzero $f \in F[x]$ of degree d, we consider the set $M_f \subseteq F^{\mathbb{N}}$ of all sequences $a \in F^{\mathbb{N}}$ that are annihilated by f. For example, M_{x^d-1} is the set of all periodic sequences with period d. M_f is an $F[x]$-submodule of $F^{\mathbb{N}}$, since $a + b$ and $g \bullet a$ are annihilated by f if a and b are and $g \in F[x]$ is arbitrary, by (1), (2), and (4). Since any sequence in M_f is completely determined by its d initial values, M_f is

https://doi.org/10.1017/CBO9781139856065.015 Published online by Cambridge University Press

d-dimensional as a vector space over F, and a basis is formed by the d shifts

$$\begin{aligned} x^0 \bullet c &= (0,0,\ldots,0,0,1,c_d,c_{d+1},\ldots),\\ x^1 \bullet c &= (0,0,\ldots,0,1,c_d,c_{d+1},c_{d+2}\ldots),\\ &\vdots\\ x^{d-1} \bullet c &= (1,c_d,\ldots,c_{2d-4},c_{2d-3},c_{2d-2},c_{2d-1},c_{2d},\ldots) \end{aligned}$$

of the **impulse response sequence** $c = (c_i)_{i\in\mathbb{N}}$ for f, whose d initial values are $0,0,\ldots,0,1$, and whose remaining values are determined by the recurrence relation $f \bullet c = \mathbf{0}$. Hence M_f is the **cyclic** $F[x]$-module $F[x] \bullet c$ generated by c: if $a = \sum_{0\le j<d} g_j(x^j \bullet c) \in M_f$ is arbitrary and $g = \sum_{0\le j<d} g_j x^j \in F[x]$, then $a = g \bullet c$.

A cyclic module $M = R \bullet c$ over a ring R is isomorphic to $R/\mathrm{Ann}(c)$. This follows from the fact that $\lambda: R \longrightarrow M$ with $\lambda(g) = g \bullet c$ is a surjective homomorphism of R-modules with kernel $\mathrm{Ann}(c)$, and by the homomorphism theorem for R-modules, the map $\varphi: R/\mathrm{Ann}(c) \longrightarrow M$ given by $\varphi(g \bmod \mathrm{Ann}(c)) = g \bullet c$ is an isomorphism (see the latest edition of van der Waerden 1931, §86). This may be familiar to the reader in the case of commutative groups, where $R = \mathbb{Z}$, M is a finite cyclic group of order $n \in \mathbb{N}$, and the module operation is $g \bullet a = a^g$ for $g \in \mathbb{Z}$ and $a \in M$. Then, for any generator c of M, we have $M = \mathbb{Z} \bullet c$, $\mathrm{Ann}(c) = n\mathbb{Z}$, and in fact $\varphi: \mathbb{Z}/n\mathbb{Z} \longrightarrow M$ with $\varphi(g \bmod n) = c^g$ is an isomorphism of $\mathbb{Z}$-modules.

In our situation, $\mathrm{Ann}(c) = \langle f\rangle$. This is because clearly f annihilates c, and on the other hand no nonzero $g \in F[x]$ of degree $k < d$ satisfies $g \bullet c = \mathbf{0}$, since the $(d-1-k)$th coefficient of $g \bullet c$ is the leading coefficient of g. Thus M_f and $F[x]/\langle f\rangle$ are isomorphic as $F[x]$-modules, where the module operation on $F[x]/\langle f\rangle$ is defined by $g \bullet (h \bmod f) = (g \bmod f)(h \bmod f) = gh \bmod f$, and

$$\varphi: F[x]/\langle f\rangle \longrightarrow M_f, \quad g \bmod f \longmapsto g \bullet c \tag{9}$$

is an isomorphism.

LEMMA 12.16. *Let $A \in F^{n\times n}$, $b \in F^n \setminus \{0\}$, and $f \in F[x]$ be the minimal polynomial of the sequence $(A^i b)_{i\in\mathbb{N}} \in (F^n)^{\mathbb{N}}$. There is a surjective F–linear map $\psi: F^n \longrightarrow F[x]/\langle f\rangle$ such that for all $u \in F^n$ we have*

$$f \text{ is the minimal polynomial of } (u^T A^i b)_{i\in\mathbb{N}} \in F^{\mathbb{N}} \iff \psi(u) \text{ is a unit.} \tag{10}$$

PROOF. We have $d = \deg f \le n$. We define the F–linear map $\psi^*: F^n \longrightarrow F^{\mathbb{N}}$ by $\psi^*(u) = (u^T A^i b)_{i\in\mathbb{N}}$. Then $\psi^*(u)$ is linearly recurrent with characteristic polynomial f for all $u \in F^n$, so that $\psi^*(u) \in M_f$. On the other hand, the d vectors $b, Ab, \ldots, A^{d-1}b \in F^n$ are linearly independent over F, by the minimality of f. Hence for any sequence $a = (a_i)_{i\in\mathbb{N}} \in M_f$ there exists a $u \in F^n$ with $u^T A^i b = a_i$ for $0 \le i < d$, and $\psi^*(u) = a$ since the first d entries of both sequences agree and

https://doi.org/10.1017/CBO9781139856065.015 Published online by Cambridge University Press

both are annihilated by f. This shows that we can regard ψ^* as a surjective map from F^n onto M_f. We now set $\psi = \varphi^{-1} \circ \psi^*$, where $\varphi: F[x]/\langle f\rangle \longrightarrow M_f$ is the isomorphism (9) of cyclic $F[x]$-modules. Then ψ is surjective,

$$g \bullet \psi^*(u) = g \bullet (\varphi \circ \psi(u)) = \varphi(g \bullet \psi(u)) = \varphi((g \bmod f) \cdot \psi(u))$$

for all $g \in F[x]$ and $u \in F^n$, and

$$\begin{aligned} f = m_{\psi^*(u)} &\iff \forall g \in F[x] \quad (g \bullet \psi^*(u) = \mathbf{0} \iff f \mid g) \\ &\iff \forall g \in F[x] \quad ((g \bmod f) \cdot \psi(u) = 0 \iff g \bmod f = 0) \\ &\iff \forall h \in F[x]/\langle f\rangle \quad (h \cdot \psi(u) = 0 \iff h = 0) \\ &\iff \psi(u) \text{ is a unit,} \end{aligned}$$

since $d \geq 1$ and any nonunit is a zero divisor in $F[x]/\langle f\rangle$ (Exercise 4.14). □

LEMMA 12.17. *Let $U \subseteq F$ be finite, $A \in F^{n\times n}$, $b \in F^n \setminus \{0\}$, f be the minimal polynomial of $(A^i b)_{i\in\mathbb{N}} \in (F^n)^{\mathbb{N}}$, and $d = \deg f$. Then the probability p that f is the minimal polynomial of the sequence $(u^T A^i b)_{i\in\mathbb{N}}$ for a $u \in U^n$ chosen uniformly at random satisfies $p \geq 1 - d/\#U$.*

PROOF. Let $\psi: F^n \longrightarrow F[x]/\langle f\rangle$ be as in Lemma 12.16, $e_j \in F^n$ the jth unit vector for $1 \leq j \leq n$, and $u = (u_1, \ldots, u_n)^T = u_1 e_1 + \cdots + u_n e_n \in F^n$ be an arbitrary vector. Since ψ is F–linear, we have

$$\psi(u) = u_1\psi(e_1) + u_1\psi(e_1) + \cdots + u_n\psi(e_n) = (u_1 h_1 + \cdots + u_n h_n) \bmod f,$$

with $h_1, \ldots, h_n \in F[x]$ of degree less than d and $\psi(e_j) = h_j \bmod f$ for all j. Let $y_1, \ldots, y_n$ be new indeterminates over $F[x]$ and $r = \operatorname{res}_x(y_1 h_1 + \cdots + y_n h_n, f) \in F[y_1, \ldots, y_n]$. Then the total degree of r is at most d, and by Lemma 6.25

$$\psi(u) \text{ is a unit} \iff \gcd(u_1 h_1 + \cdots + u_n h_n, f) = 1 \iff r(u_1, \ldots, u_n) \neq 0. \quad (11)$$

Since ψ is surjective and $d \geq 1$, there exists some $u \in F^n$ such that $\psi(u) = 1$ is a unit, and hence r is not the zero polynomial. By (10) and (11), p is the probability that $r(u_1, \ldots, u_n) \neq 0$ for $u_1, \ldots, u_n \in U$ chosen uniformly at random and independently, and the assertion follows from Lemma 6.44. □

Lemma 12.17 gives a good lower bound on the success probability of Algorithm 12.13 for fields of sufficiently large cardinality, say at least $2n$.

THEOREM 12.18.
The expected cost of Algorithms 12.12 and 12.13 is at most $2n\,c(A) + O(n^2)$ field operations if F has at least $2n$ elements.

https://doi.org/10.1017/CBO9781139856065.015 Published online by Cambridge University Press

PROOF. From Lemmas 12.16 and 12.17, we conclude that the expected number of iterations of Algorithm 12.13 is at most 2 if we take any subset $U \subseteq F$ of cardinality at least $2n$. We may assume that the vectors $Ab, A^2b, \ldots, A^{2n-1}b$ computed in step 3 of Algorithm 12.13 are stored, and then the cost for step 2 of Algorithm 12.12 is only another $O(n^2)$ field operations for vector additions and scalar multiplications. The claim now follows from Theorem 12.15. □

For "small" finite fields $\mathbb{F}_q$, we might make a suitable field extension (Exercise 12.16), which would add a factor of $O(\mathsf{M}(\log_q n))$ to the timings from Theorem 12.18. Wiedemann (1986) shows that this factor can be replaced by 2 (Exercise 12.18), by computing the least common multiple of the minimal polynomials of $(u^T A^i b)_{i\in\mathbb{N}}$ for several independently chosen uniformly random $u \in F^n$.

Wiedemann (1986) also addresses the singular and nonsquare case. A different variant in the singular case is from Kaltofen & Saunders (1991). They prove the following theorem.

THEOREM 12.19.
Let F be a field, $n \in \mathbb{N}_{>0}$, $A \in F^{n\times n}$ of rank $r \leq n$ with leading principal $r \times r$ submatrix nonsingular, and $b \in F^n$ such that the linear system $Ay = b$ is solvable. Then for any vector $v \in F^n$ there is a unique $v^* \in F^n$ such that $A \cdot (v^* - v) = b$ and the lower $n - r$ coordinates of v^* are zero. Moreover, $v^* - v$ is a uniform random vector in the solution space $\{y \in F^n : Ay = b\}$ if v is a uniform random vector in F^n.

Thus given A, b as in the theorem, we choose a random vector v and apply Wiedemann's algorithm 12.12 to the linear system $A_r y_r = b_r$, where $A_r \in F^{r\times r}$ is the leading principal submatrix of A, the vector $b_r \in F^r$ consists of the upper r coordinates of $b + Av$, and $y_r \in F^r$ is to be computed. If we let $v^* \in F^n$ be the vector whose upper part is y_r and whose lower part is zero, then the theorem implies that $y = v^* - v$ is a uniform random solution of the linear system $Ay = b$. (This works, in particular, for $b = 0$.) Since we may apply A_r to a vector $v_r \in F^r$ by applying A to the vector whose upper r coordinates are those of v_r and whose lower $n - r$ ones are zero and taking the upper r coordinates of the result, the cost for the method described above is $O(n(c(A) + n))$ operations in F.

Kaltofen & Saunders also give a probabilistic algorithm which transforms an arbitrary matrix $C \in F^{n\times n}$ into the form required in Theorem 12.19, thereby preserving the black box property, and determines its rank. If the field has at least $3(n^2 + n)$ elements, then their algorithm uses $O(n(c(C) + \mathsf{M}(n)))$ field operations and returns the correct result with probability at least $1/2$. To find a solution of the linear system with coefficient matrix C, the above method is applied to the transformed matrix, and a uniform random solution of the original linear system can be computed within the same time bound. The cost for one application of the transformed

https://doi.org/10.1017/CBO9781139856065.015 Published online by Cambridge University Press

matrix to a vector is $c(C) + 2\mathsf{M}(n)$, and the overall cost is $O(n(c(C) + \mathsf{M}(n)))$ operations.

An important aspect of Wiedemann's algorithm is that the only use of the matrix A is to evaluate it at vectors. Thus instead of storing an array of n^2 entries, all we need is a "black box" for the evaluation of A, that is, a subroutine which on input $v \in F^n$ returns $Av \in F^n$. This leads to a new way of doing linear algebra, **black box linear algebra** (or implicit linear algebra), in contrast to the traditional explicit linear algebra, where all entries of A are explicitly stored. A somewhat intermediate concept is **sparse linear algebra**, where A is stored in a sparse format, listing only those (i, j, a_{ij}) with $a_{ij} \neq 0$; this is appropriate for Wiedemann's original application to integer factorization (Chapter 19).

As an example, let $\omega \in F$ be a primitive nth root of unity and suppose that $A = \mathrm{VDM}(1, \omega, \omega^2, \ldots, \omega^{n-1})$ is the matrix of the Discrete Fourier Transform DFT_ω (Section 8.2). Thus solving $Av = b$ for v corresponds to interpolating at the powers of ω with values determined by b, and evaluation of A at v corresponds to computing the Discrete Fourier Transform of the polynomial corresponding to v, which can be done at a cost of $O(n \log n)$ arithmetic operations. Actually, this does not yield improved algorithms here, since DFT_ω^{-1} can be computed with $O(n \log n)$ operations, while the black box linear algebra approach takes $O(n^2 \log n)$, but it does when applied to the Berlekamp matrix for factoring polynomials (Section 14.8).

Notes. 12.1. Algorithm 12.1 is due to Winograd (1971), and the current world record $\omega < 2.3727$ is from Williams (2011), after the result of Coppersmith & Winograd (1990) had stood for a quarter century. The entries of the following table indicate the approximate date of discovery of new feasible matrix multiplication exponents in history; publication was often years later.

Strassen 1968	2.808	Coppersmith & Winograd 1980	2.498
Pan 1978	2.781	Strassen 1986	2.479
Bini *et al.* 1979	2.780	Coppersmith & Winograd 1986	2.376
Schönhage 1979	2.548	Williams 2011	2.373
Pan 1979	2.522		

The details of these algorithms are beyond the scope of this text. The most comprehensive treatment is in Bürgisser, Clausen & Shokrollahi (1997); we also refer to the books by Pan (1984) and de Groote (1987), and the survey articles of Strassen (1984, 1990) and von zur Gathen (1988) for details and references. Also for the rest of this section, we refer the reader to those texts.

Fast algorithms (or reductions) for the problems at the end of the section are in van der Waerden (1938), Strassen (1969, 1973a), Bunch & Hopcroft (1974), Baur & Strassen (1983), and Keller-Gehrig (1985). Chou, Deng, Li & Wang (1995) report on a successful parallel implementation of Strassen's matrix multiplication.

Various normal forms of matrices are important tools in linear algebra, say over a Euclidean domain R. The **Hermite normal form** (Notes 4.5) is particularly useful for solving

https://doi.org/10.1017/CBO9781139856065.015 Published online by Cambridge University Press

linear equations over R. Algorithm 16.26 computes the Hermite normal form over $\mathbb{Z}$ of a nonsingular square matrix, and faster algorithms are in Storjohann (2000). Giesbrecht, Storjohann & Villard (2003) is a survey of this active area by three of the main contributors.

12.2. Algorithm 12.3 is from Brent & Kung (1978). It can be slightly speeded up by using faster algorithms for *rectangular matrix multiplication*. The direct approach for multiplying an $n \times n$ matrix by an $n \times n^2$ matrix using $\omega < 2.373$ takes $O(n^{3.373})$ ring operations. Huang & Pan (1998) have improved the exponent for this particular rectangular problem to less than 3.334, and then the $n^{1.687}$ in Corollary 12.5 drops to $n^{1.667}$. In the special case $f = x^n$, Brent & Kung give a $O(\mathsf{M}(n)(n \log n)^{1/2})$ solution (Exercise 12.4). Bernstein (1998a) presents a faster algorithm for rings of small characteristic.

12.3. Linearly recurrent sequences were already studied by de Moivre, and the equivalence between finding recurrence relations and computing Padé approximants (Lemma 12.8) was known to Kronecker (1881a), page 566. The problem is also intimately connected to solving Toeplitz (or Hankel) systems of equations. Krylov (1931) invented his method in the context of solving differential equations for oscillation problems. The *transposition principle* says that for a given square matrix A, the cost of computing Av or wA, for input vectors v and w, are essentially the same (Fiduccia 1972b, 1973, Kaminski, Kirkpatrick & Bshouty 1988). Kaltofen (2000) proposes as an open problem the task to relate these two costs more precisely. The connection between these two problems is well-studied in digital filter design; see Antoniou (1979), §4.7.

12.4. The *block Wiedemann* method, proposed by Coppersmith (1994) and analyzed and improved by Kaltofen (1995b) and Villard (1997), reduces the number of evaluations of A from $2n$ to $(1+\varepsilon)n$ for any $\varepsilon > 0$. It is particularly well suited for $F = \mathbb{F}_2$ and also leads to efficient parallel algorithms over arbitrary fields.

A different black box method, based on inner products, for solving linear systems is due to Lanczos (1952). LaMacchia & Odlyzko (1990) introduced the algorithm into computer algebra by employing it for integer factorization. Block variants were given by Coppersmith (1993) and Montgomery (1995), and Eberly & Kaltofen (1997) analyze randomized Lanczos algorithms. Giesbrecht, Lobo & Saunders (1998) solve the problem of certifying inconsistency of a system of linear equations over a field or over the integers by the Wiedemann method.

Exercises.

12.1 Prove that Algorithm 12.1 works correctly.

12.2 Let $g = h = x^3 + 2x^2 + 3x + 4$ and $f = x^4 - 1$ in $\mathbb{F}_5[x]$. Trace Algorithm 12.3 on computing $g(h) \bmod f$.

12.3 Let R be a ring (commutative, with 1), $f, g, h \in R[x]$ with $\deg f = n$, $\deg g < d$, and $\deg h < m$, such that f is monic and d is a power of 2. Devise a divide-and-conquer algorithm for computing $g(h) \bmod f$ by splitting g into two blocks of size $d/2$. Prove that your algorithm takes $O(\mathsf{M}(n) \log n)$ operations in R if $dm \leq n$, and $O((dm/n)\mathsf{M}(n) \log n)$ in general.

12.4* Let $n, m \in \mathbb{N}$, R be a (commutative) ring such that $n!$ is a unit in R, and $g, h \in R[x]$ of degrees less than n. We let $k = \lceil n/m \rceil$ and write $h = h_1 + h_0$, with $h_0, h_1 \in R[x]$ such that $\deg h_0 < m$ and x^m divides h_1.

(i) Prove that the following Taylor expansion holds:

$$g(h) \equiv g(h_0) + g'(h_0)h_1 + \cdots + \frac{g^{(k)}(h_0)}{k!} h_1^k \bmod x^n.$$

https://doi.org/10.1017/CBO9781139856065.015 Published online by Cambridge University Press

(ii) The chain rule implies that $g^{(i+1)}(h_0)\cdot h_0' = (g^{(i)}(h_0))'$ for all $i\in\mathbb{N}$. Assuming that $h_0'(0)$ is nonzero, show that $g^{(i+1)}(h_0)$ rem $x^{n+k-i-1}$ can be computed from $g^{(i)}(h_0)$ rem x^{n+k-i} using $O(\mathsf{M}(n))$ operations in R, for $0\le i<k$.

(iii) Consider Brent & Kung's (1978) algorithm for computing $g(h)$ mod x^n.

ALGORITHM 12.20 Composition modulo powers of x.
Input: $n\in\mathbb{N}$ and $g,h\in R[x]$ of degrees less than n such that $h'(0)\neq 0$.
Output: $g(h)$ rem $x^n\in R[x]$.

1. write $h = h_1 + h_0$ as above
   for $i = 2,\ldots,k$ compute $h_1^i/i!$ rem x^n
2. **call** the algorithm from Exercise 12.3 to compute $g(h_0)$ rem x^{n+k}
3. **for** $i = 1,\ldots,k$ compute $g^{(i)}(h_0)$ rem x^{n+k-i}
4. **return** $\displaystyle\sum_{0\le i\le k} g^{(i)}(h_0)\frac{h_1^i}{i!}$ rem x^n

Its correctness follows from (i). Use (ii) to prove that the algorithm takes $O((k+m\log n)\mathsf{M}(n))$ operations in R.

(iv) Which choice of m minimizes the running time?

(v) Can you remove the restriction that $h'(0)$ be nonzero, using essentially the same time bound?

12.5 Prove properties (1) through (6) of a module operation $\bullet$.

12.6 Let V be a vector space over a field F.

(i) What is the minimal polynomial of the sequence $(a_i)_{i\in\mathbb{N}}\in F^{\mathbb{N}}$ defined by $a_i = 1$ for $0\le i<n$ and $a_i = 0$ otherwise?

(ii) How are m_a and $m_{x^n\bullet a}$ for $a\in V^{\mathbb{N}}$ and $n\in\mathbb{N}$ related?

12.7 Prove Lemma 12.8 (i).

12.8 Determine a recurrence relation and sufficiently many initial values of the sequence $(a_i)_{i\in\mathbb{N}}\in\mathbb{Q}^{\mathbb{N}}$ if $h = \sum_{i\ge 0} a_i x^i\in\mathbb{Q}[[x]]$ is

(i) $h = \dfrac{x^2+x}{x^3-x-1}$, (ii) $h = \dfrac{x^2-x}{x^4-x^2-x}$, (iii) $h = \dfrac{x^4+x}{x^3-x-1}$.

12.9 Compute the minimal polynomial of the sequence $1,3,4,7,11,18,29,47,\ldots$ of rational numbers using Algorithm 12.9. You may assume that the recursion order is at most four. Give the next 12 elements of the sequence.

12.10* Let F be a field, $f\in F[x]$ an irreducible polynomial of degree n, $E = F[x]/\langle f\rangle$, $\alpha = x$ mod $f\in E$, and $\beta = g(\alpha)\in E$ for some nonzero polynomial $g\in F[x]$ of degree less than n.

(i) Prove that the minimal polynomial $m\in F[x]$ of β over F is equal to the minimal polynomial of the sequence $(\beta^i)_{i\in\mathbb{N}}\subseteq E^{\mathbb{N}}$ over F.

(ii) Let $\tau: E\longrightarrow F$ be the F-linear map with $\tau(\sum_{0\le i<n} c_i\alpha^i) = c_0$ for all $c_0,\ldots,c_{n-1}\in F$. Prove that m is the minimal polynomial of the sequence $(\tau(\beta^i))_{i\in\mathbb{N}}\subseteq F^{\mathbb{N}}$. Hint: m is irreducible.

(iii) Show that m can be computed using $O(n\mathsf{M}(n))$ operations in F.

(iv) Compute the minimal polynomial of $2^{2/3}+2^{1/3}+1$ over $\mathbb{Q}$.

(Shoup (1999) employs Algorithm 12.3 to compute minimal polynomials over a finite field F with $O(n^{1/2}\mathsf{M}(n)+n^2)$ arithmetic operations.)

12.11 Let F be a field, $A\in F^{n\times n}$, $u,b\in F^n$, and define sequences $a = (A^i)_{i\in\mathbb{N}}$ and $a^* = (A^ib)_{i\in\mathbb{N}}$.

(i) Prove that $f\in F[x]$ is a characteristic polynomial of a if and only if $f(A) = 0$ in $F^{n\times n}$.

(ii) Prove that $f\in F[x]$ is a characteristic polynomial of a^* if and only if $f(A)b = 0$ in F^n.

https://doi.org/10.1017/CBO9781139856065.015 Published online by Cambridge University Press

12.12* This continues Exercise 12.11. Let $a^{**} = (u^T A^i b)_{i\in\mathbb{N}}$. Find a situation where $u^T f(A)b = 0$, but f is not a characteristic polynomial of a^{**}. Can you determine a stronger condition such that a similar equivalence as in Exercise 12.11 holds? Hint: Consider the condition that $u^T f(A)$ be orthogonal to the Krylov subspace $\langle A^i b : i \in \mathbb{N}\rangle$ of F^n generated by b.

12.13⟶ Let

$$A = \begin{pmatrix} 1 & 2 & 3 \\ 4 & 0 & 1 \\ 1 & 3 & 1 \end{pmatrix} \in \mathbb{F}_5^{3\times 3}, \quad b = \begin{pmatrix} 0 \\ 1 \\ 2 \end{pmatrix} \in \mathbb{F}_5^3,$$

and compute $A^{-1}b$ using Wiedemann's algorithm 12.12.

12.14* Let F be a field, $n \in \mathbb{N}$, $A \in F^{n\times n}$, and $e_i \in F^n$ be the ith unit vector, that is, the column vector with 1 in coordinate i and 0 everywhere else, for $1 \le i \le n$. Prove that if $f, f_1, \ldots, f_n \in F[x]$ are the minimal polynomials of A, $(A^i e_1)_{i\in\mathbb{N}}, \ldots, (A^i e_n)_{i\in\mathbb{N}}$, respectively, then $f = \operatorname{lcm}\{f_1, \ldots, f_n\}$. Generalize this to arbitrary bases $e_1, \ldots, e_n$ of F^n.

12.15* Let F be a field.

(i) Design an algorithm that, given a matrix $A \in F^{n\times n}$, computes its minimal polynomial by randomly choosing $u, b \in F^n$, computing the minimal polynomial of $(u^T A^i b)_{i\in\mathbb{N}}$, and checking whether it is actually the minimal polynomial of A. Prove that your algorithm is correct if it halts.

(ii) Let $f \in F[x]$ be the minimal polynomial of A. Show that there is a surjective bilinear map $\psi : F^n \times F^n \longrightarrow F[x]/\langle f\rangle$ (bilinear means that ψ is linear with respect to both arguments) such that

$$f \text{ is the minimal polynomial of } (u^T A^i b)_{i\in\mathbb{N}} \in F^n \iff \psi(u,b) \text{ is a unit}$$

for all $u, b \in F^n$. Hint: Proceed as in Lemma 12.16.

(iii) Use (ii) to show that the probability for f being the minimal polynomial of $(u^T A^i b)_{i\in\mathbb{N}}$, when the entries of u and b are chosen uniformly at random and independently from a fixed finite subset $U \subseteq F$, is at least $1 - 2\deg f/\#U$. Hint: Proceed as in the proof of Lemma 12.17.

12.16 Let $F \subseteq E$ be fields, $n \in \mathbb{N}$, $V = F^n$, $b \in V$, and $A \in F^{n\times n}$.

(i) Let $a = (A^i b)_{i\in\mathbb{N}} \in V^{\mathbb{N}}$. Prove that the recursion order of a is the smallest $r \in \mathbb{N}$ such that $b, Ab, \ldots, A^r b$ are linearly dependent in F^n.

(ii) Let $r \le n$, and prove that vectors $b_0, \ldots, b_{r-1} \in F^n$ are linearly dependent in F^n if and only if they are linearly dependent in E^n. Hint: Gaussian elimination. Conclude that the minimal polynomial of a over F is the same as over E.

12.17* Let $n \in \mathbb{N}$, F be a field of cardinality at least $4n$, and $A \in F^{n\times n}$. Give a probabilistic "black box" algorithm taking $O(c(A)n + n^2)$ field operations that decides whether A is singular, such that the answer is always correct when A is nonsingular and correct with probability at least $1/2$ when A is singular. Hint: Exercise 12.15.

12.18** Let F be a field, $n \in \mathbb{N}$, $A \in F^{n\times n}$, and $b \in F^n$. This exercise discusses a modification of Algorithm 12.12, due to Wiedemann (1986), which proves that the time bound from Theorem 12.18 can be achieved within a factor of 2 when F is a small finite field.

(i) Let V be a vector space over F, $f \in F[x]$ the minimal polynomial of a linearly recurrent sequence $a \in V^{\mathbb{N}}$, and $g \in F[x]$. Prove that $g \bullet a$ has minimal polynomial $f/\gcd(f, g)$.

(ii) Replace steps 1, 2, and 5 of Algorithm 12.13 by

1. let $U \subseteq F$ be finite, $\quad g \longleftarrow 1$
2. **if** $b = 0$ **then return** g
5. $g \longleftarrow gm, \quad b \longleftarrow m(A)b, \quad$ **goto** 2

Prove that the modified algorithm works correctly.

https://doi.org/10.1017/CBO9781139856065.015 Published online by Cambridge University Press

(iii) Let $u_k \in F[x]$ be the value of u chosen in the kth iteration and $g_k \in F[x]$ the minimal polynomial of the sequence $a^{(k)} = (u_k^T A^i b^*)_{i\in\mathbb{N}}$, where $b^* \in F^n$ is the initial value of b. Prove that the invariant $m = g_k/\gcd(g, g_k)$ holds after the kth pass through step 3, and conclude that $g = \operatorname{lcm}(g_1, \ldots, g_k)$ holds before the kth pass through step 2.

(iv) Let $U = F = \mathbb{F}_q$ be a finite field with q elements and $f \in F[x]$. Prove that for k polynomials $h_1, \ldots, h_k \in F[x]$ of degree less than $\deg f$ chosen uniformly at random and independently, the probability that $\gcd(h_1, \ldots, h_k, f) = 1$ is $p_k = \prod_{1\le j\le r}(1 - q^{-k\deg f_j})$, where $f_1, \ldots, f_r \in F[x]$ are the distinct monic irreducible factors of f.

(v) Now let f be the minimal polynomial of $(A^i b^*)_{i\in\mathbb{N}}$. Prove the following generalization of Lemma 12.16 (with b replaced by b^*): If $\psi(u) = h \bmod f$ with $h \in F[x]$, then $m_{\psi^*(u)} = f/\gcd(f, h)$. Conclude that p_k from (iv) is the probability that the above algorithm terminates after at most k iterations.

(vi) Let $n_i = \#\{1 \le j \le r : \deg f_j = i\}$ for all i. Prove that

$$\prod_{1\le j\le r}(1 - q^{-k\deg f_j}) \ge 1 - \sum_{1\le j\le r} q^{-k\deg f_j} = 1 - \sum_{i\ge 1} n_i q^{-ki}.$$

Use that $n_i \le q^i/i$, by Lemma 14.38, to show that $p_k \ge 1 - 2q^{1-k}$ if $k \ge 2$. Conclude that the expected number $\sum_{k\ge 0}(1 - p_k)$ of iterations of the algorithm is at most 4.

Research problem.

12.19 Can you improve the cost for modular composition of polynomials of degrees at most n to, say, $O^{\sim}(n^{1.5})$ or better?

https://doi.org/10.1017/CBO9781139856065.015 Published online by Cambridge University Press

https://doi.org/10.1017/CBO9781139856065.015 Published online by Cambridge University Press

Völker, hört die Signale![1]

Emil Luckhardt (c. 1890)

L'étude approfondie de la nature est la source la plus féconde des découvertes mathématiques. Non seulement cette étude, en offrant aux recherches un but déterminé, a l'avantage d'exclure les questions vagues et les calculs sans issue: elle est encore un moyen assuré de former l'Analyse elle-même, et d'en découvrir les éléments qu'il nous importe le plus de connaître, et que cette science doit toujours conserver.[2]

Jean Baptiste Joseph Fourier (1822)

Unsere Allergrößten, wie Archimedes, Newton, Gauß, haben stets Theorie und Anwendungen gleichmäßig umfaßt.[3]

Felix Klein (1908)

Die Mathematiker sind eine Art Franzosen, redet man zu ihnen, so übersetzen sie es in ihre Sprache und dann ist es alsobald ganz etwas anders.[4]

Johann Wolfgang von Goethe (1829)

He said he would rather decline two drinks than one German adjective.

Mark Twain (1879)

[1] Peoples, hear the signals!

[2] The deep study of nature is the most fruitful source of mathematical discoveries. By offering a well-defined goal for research, this study not only has the advantage of excluding vague questions and useless calculations, but is also a sure means of developing mathematics itself and of discovering the elements which are most important to know, and which this science ought always to conserve.

[3] Our greatest mathematicians, as Archimedes, Newton, and Gauß, always united theory and applications in equal measure.

[4] Mathematicians are like Frenchmen; if you talk to them, they translate it into their own language, and right away it is something entirely different.

https://doi.org/10.1017/CBO9781139856065.015 Published online by Cambridge University Press